

DISTANCE IN KNN (OTHER DISTANCE METRICS)

Manhattan Distance (L1 norm)– Useful in high dimensional spaces, especially when dimensions are not similar in type (e.g., height vs. age). It is less influenced by outlier features.

► Formula: $d(p, q) = |p_1 - q_1| + |p_2 - q_2| + \dots + |p_n - q_n|$

► Python Snippet:

```
dst = distance.cityblock(p, q)
#or
knn = KNeighborsClassifier(n_neighbors=3, metric='manhattan')
```

DISTANCE IN KNN (OTHER DISTANCE METRICS)



Hamming Distance– It is tailored for string comparison, especially in the context of binary data (e.g., 0s and 1s). It quantifies **dissimilarity** by counting differing symbol positions (bits) between two sequences. This metric is particularly well-suited for binary classification tasks and when categorical attributes are encoded as binary strings, such as through **one-hot encoding**.

► Formula: $d(p, q) = \sum_{i=1}^n |p_i - q_i|$, where p_i and q_i can take values 0 or 1.

► Python Snippet:

```
import numpy as np

p = np.array([1, 0, 1, 0, 1, 0, 1])
q = np.array([1, 0, 0, 1, 0, 0, 1])

hamming_dist = np.sum(p != q)
print("Hamming Distance:", hamming_dist)
#or
knn = KNeighborsClassifier(n_neighbors=3, metric='hamming')
```